板子先只接電腦

USB 線同時提供電力與傳輸程式。先看到文字，再接按鈕。

ESP32 是執行程式的開發板；本週按鈕只提供按下 / 放開訊號，計次、保存及計時由 ESP32 程式完成，結果傳回電腦。

> 開始時 USB 與全部電源未接，麵包板為空板。本例限定已核對的 YD-ESP32-S3 Type-A V1.5 / N16R8；先移除電池、杜邦線及其他模組。板子放在乾燥、不導電平面；板型或印字不同先核對，不照位置猜。



課堂實物背面。依 COM 印字找接頭；翻面後不要靠上下位置猜。

1. 開啟 Week1 已安裝的 Arduino IDE（編輯、編譯及上傳程式的軟體），先看 **Tools → Port** 清單。
2. 用可傳資料的 USB 線連接電腦與板上 **COM** 接頭。這個接頭經 CH343 將電腦與板子串列介面連起來。
3. 等系統辨識，再打開 **Tools → Port**，選剛新增的 COM 名稱。不是每台電腦都叫 COM8。

沒有新增 Port？

先換資料線、確認 COM 接頭。仍沒有時，查看裝置管理員的 CH343 / 未知 USB 裝置，依下方廠商資料安裝驅動。驅動讓 Windows 認得 CH343；重插 USB 後，Port 應多出一個 COM 名稱。需要管理員權限時找助教。

PWR 燈亮只證明有供電，不證明資料傳輸正常。板卡廠商資料與 CH343 驅動入口。

開啟 Hello，選好板型再上傳

完整程式在獨立檔案；保留同名資料夾，之後才能重新找到它。

```
program/week2/week2_hello_serial/week2_hello_serial.ino
```

1. 從課程 GitHub 的 program/week2取得程式；下載儲存庫 ZIP 後先解壓縮。複製其中整個 **week2_hello_serial** 資料夾到自己的課程資料夾，以 **File → Open** 開啟同名 .ino。
2. **Tools → Board → esp32 → ESP32S3 Dev Module**。Port 選第 1 頁新增的名稱；每個新視窗都核對一次。

Tools 選項	本例設定
Upload Speed	115200
USB Mode	Hardware CDC and JTAG
USB CDC On Boot	Disabled
Upload Mode	UART0 / Hardware CDC
Flash Mode / Flash Size	QIO 80MHz / 16MB (128Mb)
Partition Scheme	16M Flash (3MB APP/9.9MB FATFS)
PSRAM	OPI PSRAM
Erase All Flash Before Sketch Upload	Disabled

Week1 的 esp32 by Espressif Systems 套件讓 IDE 出現 ESP32S3 板型，並能為它編譯；找不到時先回 Week1 安裝。表中 N16R8 的 Flash 保存程式，PSRAM 是執行時額外記憶體。

3. 按 **Ctrl+S**，關閉其他 Serial Monitor，點工具列**右箭頭 Upload**。勾勾 Verify 只編譯，不會送進板子。
4. 等 IDE 顯示 **Done uploading** / 上傳完成；過程不拔線。若失敗，保留錯誤文字，先查板型、Port 與 COM 接頭。

Espressif Tools 選單；Arduino 上傳操作。

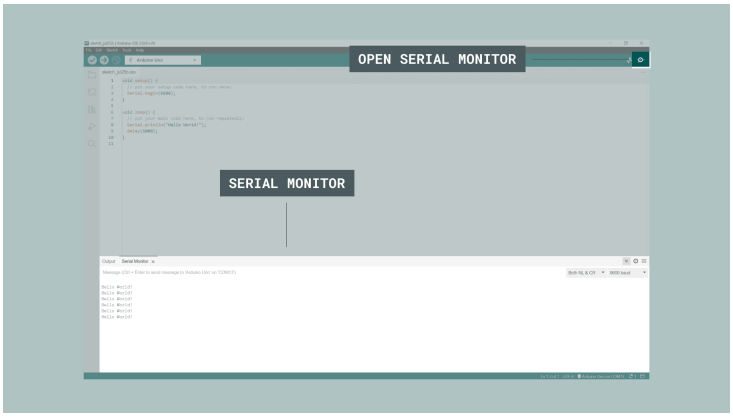
看到 Hello，再改成 Hi

Serial Monitor 是顯示板子傳回文字的視窗；它不是編譯訊息區。

1. USB 保持接著，開 **Tools → Serial Monitor**，速度選 **115200**。
2. 看 Serial Monitor 分頁。每隔約一秒應新增一行 Hello。

```
Hello
Hello
Hello
```

預期輸出示意，不是實測紀錄。



Arduino 官方舊版示例只供找入口。本課選 ESP32S3 與 115200，不照抄圖片的 UNO 或 9600。

修改測試：在自己的檔案找到 `Serial.println("Hello")`，只把引號內 Hello 改成 Hi。儲存、關閉 Monitor、重新 Upload，再開 Monitor，應持續新增 Hi。

看到的情況	先查這裡
沒有文字	確認不是 Output 分頁；選同一 Port、115200。上傳完成後可按板上 RST 重新啟動。
亂碼	核對 Monitor 速度與程式 Serial.begin 的 115200。
仍有 Hello	看最新一行，不看歷史行；確認上傳的是修改後視窗且已成功。

現在理解：setup 在開機時準備一次；loop 反覆送出文字。Upload 傳的是程式，Serial 傳回的是執行結果。完成後拔 USB，才開始接點辨認。

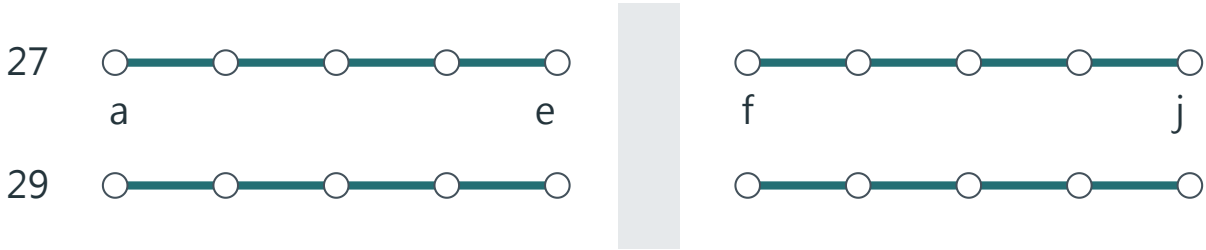
Arduino Serial Monitor；截圖未修改，Arduino docs-content，CC BY-SA 4.0。

麵包板與按鈕接點

麵包板讓導線與按鈕不用焊接就能連起來。靠近的孔不一定相通。

> 拔掉 USB 與全部電源。按鈕尚未連到開發板；通斷檔只能測無外部電源的物件。

先讀孔位：a27 是 a 欄、第 27 列。這種板的 a ~ e 同列相通，f ~ j 同列相通；中央槽隔開左右，相鄰兩列也不相通。



用通斷檔核對這幾個孔

A830L 是三用電表。本週用通斷檔，在斷電時確認兩處是否相通；相通時會嗶聲。

1. 黑色表筆**插頭插電表的 COM 插孔**，紅色表筆**插頭插 VΩmA**；10A 插孔留空。這裡的 COM 不是開發板 USB 接頭。旋鈕轉通斷蜂鳴符號；其他電表先核對說明。
2. 金屬筆尖分開應不叫，碰在一起應叫。用兩條公對公線（兩端都是金屬針）引出孔位，筆尖碰線端，不將粗表筆塞進孔。
3. 空板 a27 與 e27 應相通；a27 與 a29、a27 與 f27 應不通。結果不同先查孔位與接觸。

四腳按鈕先測，再放進麵包板

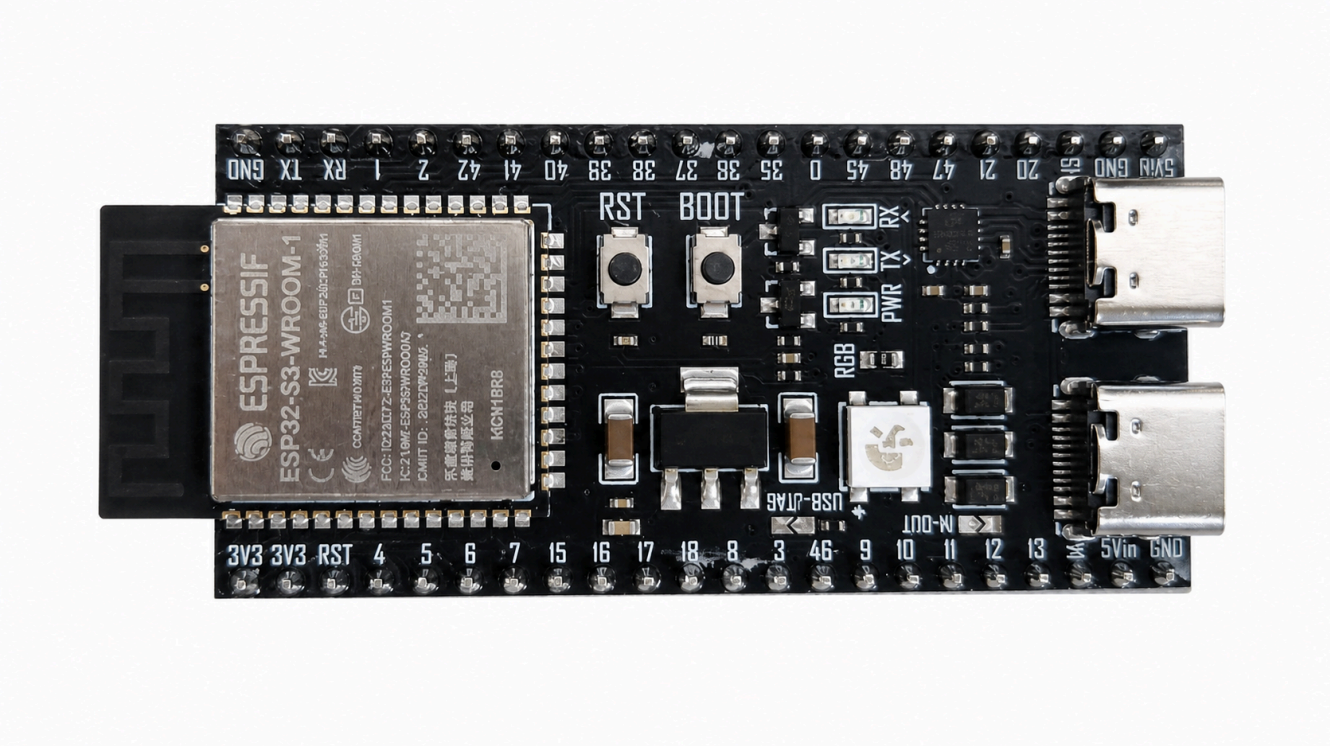
按鈕先不插麵包板，兩筆尖直接碰兩隻腳，一對一對測。放開時找出原本相通的兩腳，作一組；另兩腳作另一組。跨組兩腳應是放開不通、按下相通、再放開不通。

按鈕跨中央槽放在 **e27、f27、e29、f29**。第 6 頁圖要求第 27 列兩腳是同組，第 29 列兩腳是另一組；兩組各接一條線。腳距不合不要硬壓。

Fluke 通斷測試。蜂鳴表示兩筆間電阻夠小，不等於任何帶電情況都安全。

ESP32 接腳

USB 保持拔除；只看印字找位置，還不接線。



課堂實物正面。天線在左、兩個 USB 接頭在右。照片中的板型是 YD-ESP32-S3 Type-A V1.5 / N16R8。

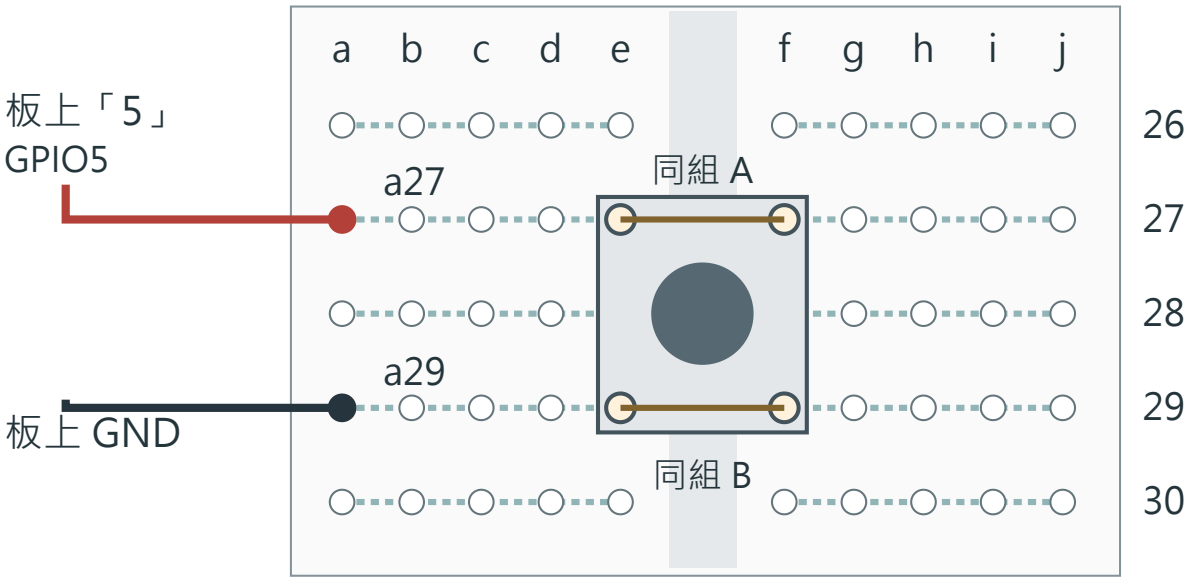
找哪個印字	位置與用途
5	下排 4 與 6 之間。GPIO 是讀取或輸出訊號的接腳；本週 GPIO5 讀取按鈕。
GND	上排最左端、靠天線。GND 是電路共同的 0 V 參考；本例按下時將 GPIO5 接到它。
RST	板上印 RST 的小按鍵；按一下重啟程式，不是 BOOT，也不是外接按鈕。

板子放在乾燥、不導電的桌面，用公對母杜邦線接向麵包板。母頭是有插孔的一端，套在板上排針；公頭有金屬針，插進麵包板。

> 不把這塊寬板硬插進單片 400 孔麵包板，以免遮住可接線孔。板子或印字與照片不同，先查對應板型資料；不得猜接 3V3、5Vin 或未知排針。

按鈕接線

板子留在桌上；用本堂剛確認接點的按鈕，新增兩條線。



四腳：e27、f27、e29、f29；中央槽左右原本不通。

兩條實線是新接線；虛線是板內連接，按鈕上的線表示固定同組。板上腳位見第 5 頁照片，圖左標籤不是排針排列。

公對母杜邦線的母頭	同一條線的公頭
開發板印字 5（GPIO5）	麵包板 a27，同列接到按鈕 e27
開發板 GND	麵包板 a29，同列接到按鈕 e29

接板前核對：按鈕裝好、尚未接板時，a27 與 j27、a29 與 j29 各自相通；a27 與 a29 放開不通、按下才通。符合後移除量測線、電表 OFF，再接上表兩線。

> USB 仍拔除。只有兩條接線；3V3、5Vin 不接，GPIO4 留空。金屬針不碰相鄰腳。這個孔位只適用已確認兩組接點的按鈕；不由外形推定同批全部相同。

按鈕狀態

沿用本堂剛才兩條接線；這次程式只是觀察狀態，還不計次。

`program/week2/week2_button_state/week2_button_state.ino`

1. 複製整個同名資料夾作工作副本，以 File → Open 開啟 .ino。
2. 接線核對完成後，插回板上 COM。依第 2 頁核對板型與 Port，關閉 Monitor，再 Upload。
3. 開 Serial Monitor，選 115200。放開約一秒、按住約一秒、再放開約一秒，慢慢比較。

手的動作	每約 0.3 秒新增的文字
放開	released
按住	pressed
再次放開	released

```
released
pressed
released
```

預期格式示意，不是實測；同一狀態可能重複多行。

確認兩種狀態，再做三件作品

按住出現多行 pressed，表示程式重複讀到按住，不表示按了多次。這支程式也不保證看見快速點按；先採上面的慢速測試。

沒有得到預期狀態	處理方式
一直 pressed	先拔 USB，查兩線是否誤接同組或同列、是否短接到 GND。
一直 released	先拔 USB，查斷線、腳沒插到底、按鈕兩組與板上 5。
完全沒字 / 還是 Hello	先查新程式是否上傳成功、Port、COM 接頭及 115200。

Q1 · 按鈕計數器

接線不變，改由程式保存次數。先放開按鈕，再上傳。

```
program/week2/week2_button_count/week2_button_count.ino
```

1. 保留本堂已成功的 GPIO5 與 GND 接線。開同名資料夾內 .ino 的工作副本，核對第 2 頁設定與 Port。
2. 關閉 Monitor，Upload 完成後再開 Monitor，選 115200。放開按鈕，按一下板上 **RST**，應看見 released count=0。
3. 按住約兩秒，次數變 1 後不再新增。放開約一秒再按，變 2；再做一次變 3。Q1 先看 count；同一程式的狀態文字留到 Q4 比較。

```
released count=0
pressed count=1
released count=1
pressed count=2
```

預期格式示意，不是實測。每次重啟從 0 開始；不清除視窗中更早的歷史行。

Q1 回答：放開後重啟，次數為 0；按住兩秒，次數從 0 變 1，按住期間一直是 1。

參考程式中的三件事

程式位置	連回剛才的操作
count	在記憶體中保存已按幾次；count++ 將它加 1。
DEBOUNCE_MS = 40	讀值連續穩定 40 毫秒才接受，避免接點短暫跳動被當成多次。這叫去抖。
readyForPress	穩定放開後才允許下次計數。一次穩定按下後立即取消允許，所以一直按住不再加。

millis() 是從程式啟動經過的毫秒數。changedAt 記錄最近一次讀值改變的時間；現在時間減去它，就是本次連續穩定多久。這裡先讀取，再判斷穩定、更新 count，最後 Serial 才把文字送回電腦。

沒有新行：按住不動時正常；若漏看啟動行，放開再按 RST。按放皆無反應，回第 7 頁同接線狀態程式排查；一按多加，先核對是否開錯舊版程式，再斷電查接觸。

Q2 · 文字開關

這件作品記住開或關，並不是讓文字一直跟著按鈕按放。

接線：與本堂計數器相同，板上 5→a27、GND→a29，按鈕 e27 / f27 / e29 / f29。無需 LED 或其他負載；若從零開始，依第 4 ~ 6 頁完整接線。

```
program/week2/week2_button_toggle/week2_button_toggle.ino
```

1. 開啟同名資料夾的工作副本，核對第 2 頁設定。關閉 Monitor、Upload，再開 Monitor 選 115200。
2. 放開後按 RST，最新一行應為 OFF。按住兩秒、放開，最新一行應為 ON 且放開不變。
3. 再按住兩秒、放開，應回 OFF；再按一次為 ON。不要用視窗中舊行判斷目前狀態。

```
OFF
ON
OFF
ON
```

預期順序示例，不是實測；每次切換才新增一行。

沒有達到預期

放開就回 OFF：先確認開的是 toggle 而非狀態顯示檔。按住反覆切換：查是否取消穩定放開才接受下一次的判斷。完全沒反應：回第 7 頁用狀態程式檢查，斷電後才調線。

短原理與 Q2 回答

`isOn` 記住 ON / OFF。接受一次新按下才改成相反狀態；放開只準備下一次按下，不改 isOn。因此第一次按下後，即使放開仍應為 **ON**。

與 Q1 相同，讀值穩定 40 ms 才接受。啟動時按住不切換，必須先放開。

Q3 · 單鍵碼表

仍用一顆按鈕和相同兩條線，不另接時鐘或顯示器。

接線：GPIO5→a27、GND→a29；四腳按鈕接法見第 6 頁。本堂可保留成功接線；不是保留上週電路。

```
program/week2/week2_stopwatch/week2_stopwatch.ino
```

1. 開工作副本，依第 2 頁設定 Upload。Monitor 選 115200，放開按鈕後按 RST。
2. 看到 STOP elapsed_ms=0。按一下後應為 RUN 且數字增加；再按一下為 STOP 且停止增加。
3. STOP 等約兩秒，再按一下；數字從原處繼續。按住兩秒只能切換一次。

```
STOP elapsed_ms=0
RUN elapsed_ms=0
STOP elapsed_ms=2100
RUN elapsed_ms=2100
```

示例省略中間每約 200 ms 更新的行。2100 是示例，手動按鍵不要求精確到毫秒。

RUN 表示計時中，STOP 表示暫停；elapsed_ms 是畫面上的累計毫秒數。程式用 running 記住是否正在計時：true 表示正在計時，false 表示暫停。

先查這些現象

STOP 數字仍增加：確認新檔已上傳，看最新行是否真的 STOP。恢復時突然多了休息時間：查是否只在 running 為 true 時累加。沒有切換則回第 7 頁查按鈕狀態。

短原理與 Q3 回答

`elapsedMs` 保存真正計時的總毫秒數，1000 ms = 1 秒。程式每次只在 RUN 時加入距離上次檢查的時間；STOP 仍更新檢查時刻，但不加入總時間，因此休息時間被排除。

三個數字依實測填：第一次 STOP、STOP 約兩秒後、再次 RUN 約一秒後的第二次 STOP。前兩個相同，第三個只多出再次 RUN 的時間；重啟回 0。這是教學碼表，不用來宣稱精密計時。

Q4 · 狀態與次數

重用 Q1 計數器；沒有新增接線，也不用改程式設定。

接線：板上 5→a27、GND→a29；按鈕跨槽接 e27 / f27 / e29 / f29。從空板開始時先依第 4 ~ 6 頁確認。

```
program/week2/week2_button_count/week2_button_count.ino
```

1. 重新開啟 Q1 工作副本，依第 2 頁設定 Upload，開 Monitor 選 115200。
2. 保持按鈕按住，按一下板上 RST；重啟後仍按住約兩秒，最新一行應是 pressed count=0。
3. 放開約一秒，應是 released count=0；再按住約兩秒，才變 pressed count=1，並保持不變。

手的動作	最新一行
按住並按 RST，仍按住兩秒	pressed count=0
放開一秒	released count=0
再按住兩秒	pressed count=1，保持不變

預期格式示例，不是實測。接點穩定約 40 ms 才改畫面；看最新一行，不把歷史行當成現在。

Q4 直接回答

重啟時是 pressed、count=0；先放開再按，才加到 1。狀態描述現在的手勢；count 保存本次啟動後接受的新按下次數。重啟清除舊次數，已按住不算新按下，所以 pressed 也可以同時是 0。

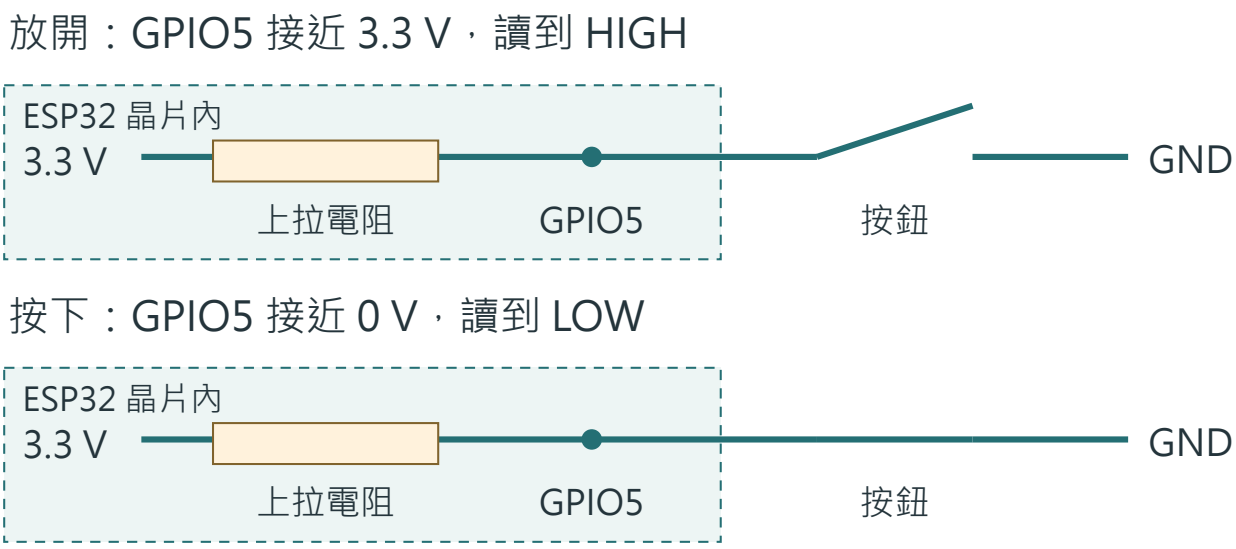
程式先讀按鈕、等待穩定，再更新 count，最後把狀態與次數傳到 Serial Monitor。傳出一行文字不會另外增加次數。

反應不符：回第 7 頁確認按放文字。狀態不變先斷電查兩組接點；狀態正確但連加，查是否開錯檔或取消了穩定放開才接受下一次的條件。

按鈕上拉電阻

GPIO 是晶片讀取或輸出訊號的接腳；本週 GPIO5 讀取按鈕。

程式用 `INPUT_PULLUP` 啟用晶片內的上拉電阻，讓 GPIO5 經電阻連到 3.3 V。不需在麵包板再加一顆電阻。電壓比較 GPIO5 與 GND；GND 是 0 V 參考，V 是伏特。



正常接法的兩種狀態。虛框是晶片內部，不是實物排針位置；電壓為近似說明，不是本組實測。

放開讀 HIGH：通往 GND 的路斷開，輸入經電阻連到 3.3 V；輸入取用極小電流，電阻兩端電壓差很小。

按住讀 LOW：GPIO5 直接連到 GND，接近 0 V。3.3 V → 上拉電阻 → 按鈕 → GND 是電流路徑，電阻限制電流。程式將 LOW 顯示為 pressed，HIGH 顯示為 released；次數由另一個變數保存。

紙上反例：如果取消上拉，放開不能保證 LOW。通往 GND 與 3.3 V 的路都斷開，輸入沒有可靠電壓，稱為浮接，可能受干擾。取消上拉是斷開支路，不是用導線取代電阻。

> 保留正常 INPUT_PULLUP 設定；不要在帶電時取消上拉或拔線。用導線取代電阻可能讓電源與 GND 短接，也不在實物上測試。

> **下課收納：**保存程式，拔 USB 與全部電源，再拆除所有導線、按鈕及元件，麵包板恢復空板。

Espressif GPIO；Arduino Debounce。本冊示例不是新版硬體實測紀錄。